ECOLE NATIONALE
SUPÉRIEURE
D'INFORMATIQUE

People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research

Intelligent and Communicating Systems, ICS

2nd Year Specialty SIQ G02

Supplementary Laboratory Report

ESP32 IoT Device Control with TUYA - Detailed Configuration Guide

Academic Year 2024–2025

Realised by:

- SADOUN Amel
- SALHI Fatma
- CHEKMAN Meyssem
- LADOUL Mahdi
- HAMROUN Mohamed Ali
- BENAHMED Lyna Kamilia

Supervised by: Dr. Abdenour SEHAD
a_sehad@esi.dz

Chapter 1

Configuration Steps

This document serves as a detailed graphical reference for **Part II - Cloud Control** of the main report. It contains all the step-by-step screenshots and instructions necessary to reproduce the device creation and firmware compilation process on the Tuya IoT Platform. Each step explains the purpose and any important parameters.

1.1 Device Registration on Tuya IoT Platform

The following steps were performed to register the custom ESP32 device and obtain the necessary cryptographic keys:

1. **Initiating Product Creation:**
   Open the Tuya Developer Console in a web browser. Navigate to the **Product Development** section, then click the **Create** button to start a new product. This step initializes a new device in your Tuya account and prepares it for custom firmware assignment.

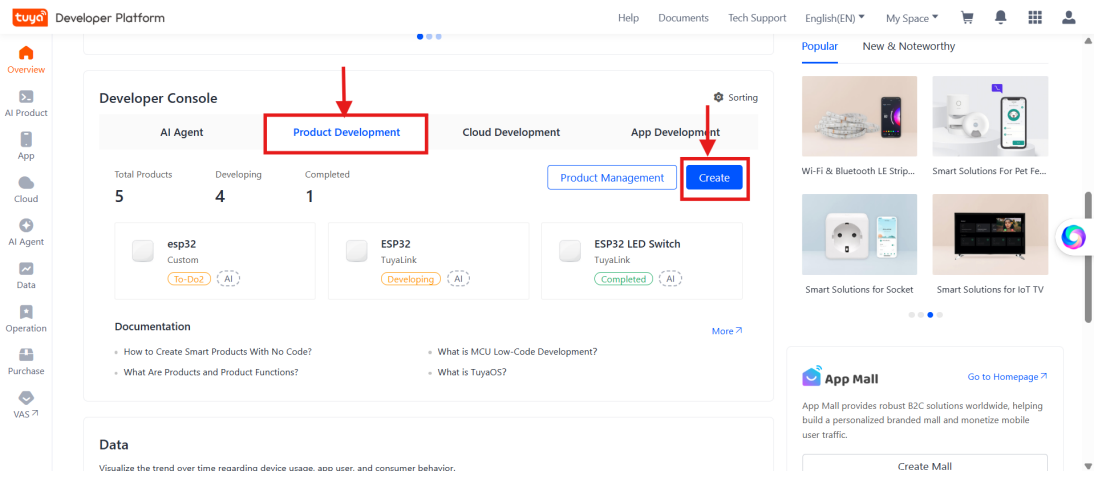

Figure 1.1: Tuya Developer Console dashboard where you can create a new product.

2. **Solution Selection:**
   In the product creation wizard, select **Custom Solution** under the "Others" tab. Then choose the **TuyaOS** development mode. This ensures that you can upload your own ESP32 firmware instead of using the prebuilt Tuya app solutions.

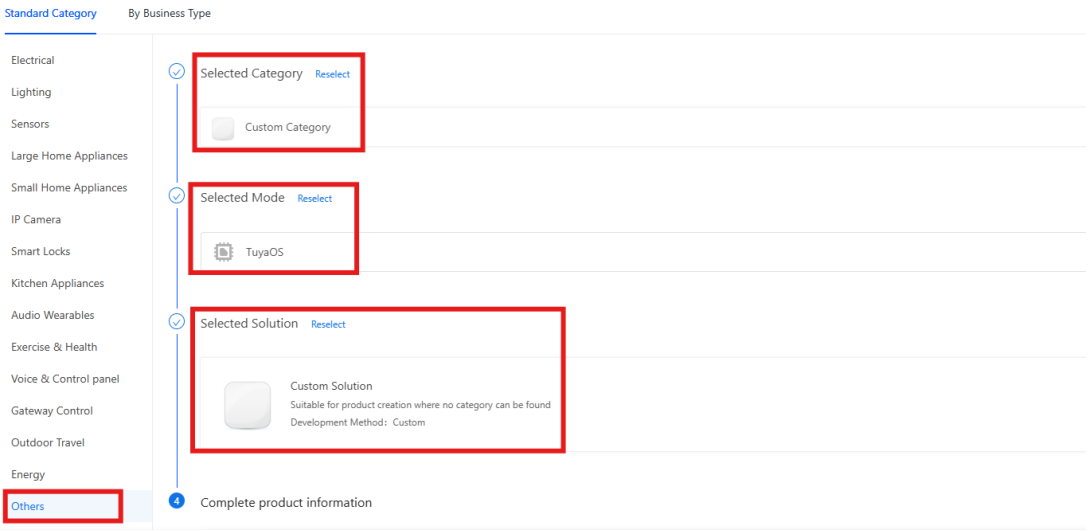

Figure 1.2: Selecting Custom Solution and TuyaOS mode for custom firmware development.

3. **Product Definition:**
   Enter a descriptive name for your device (e.g., `esp32_firmware`). Select the Wi-Fi protocol and enable low power mode if required. This step determines the connectivity and power parameters of your device. Make sure the protocol matches the ESP32 module you are using.

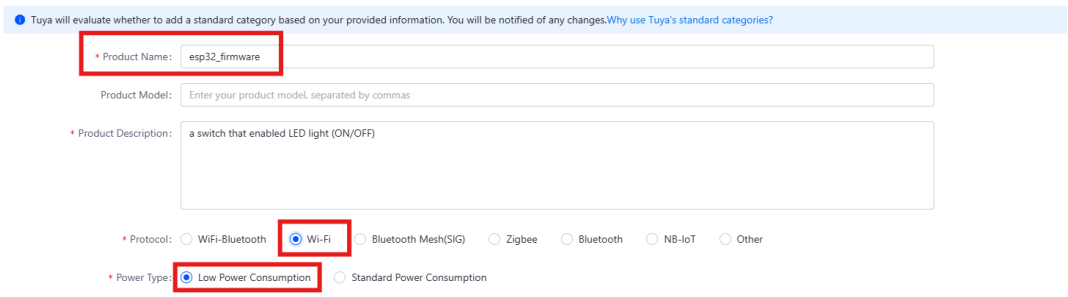

Figure 1.3: Defining basic product information such as name, Wi-Fi type, and power options.

4. **Function Definition:**
   Add the standard **Light Action** function to your product. This maps the cloud-controlled switch to a boolean data point (DP) on the ESP32. Verify that the data type is Boolean, as this will be read by the firmware to turn the device on/off.

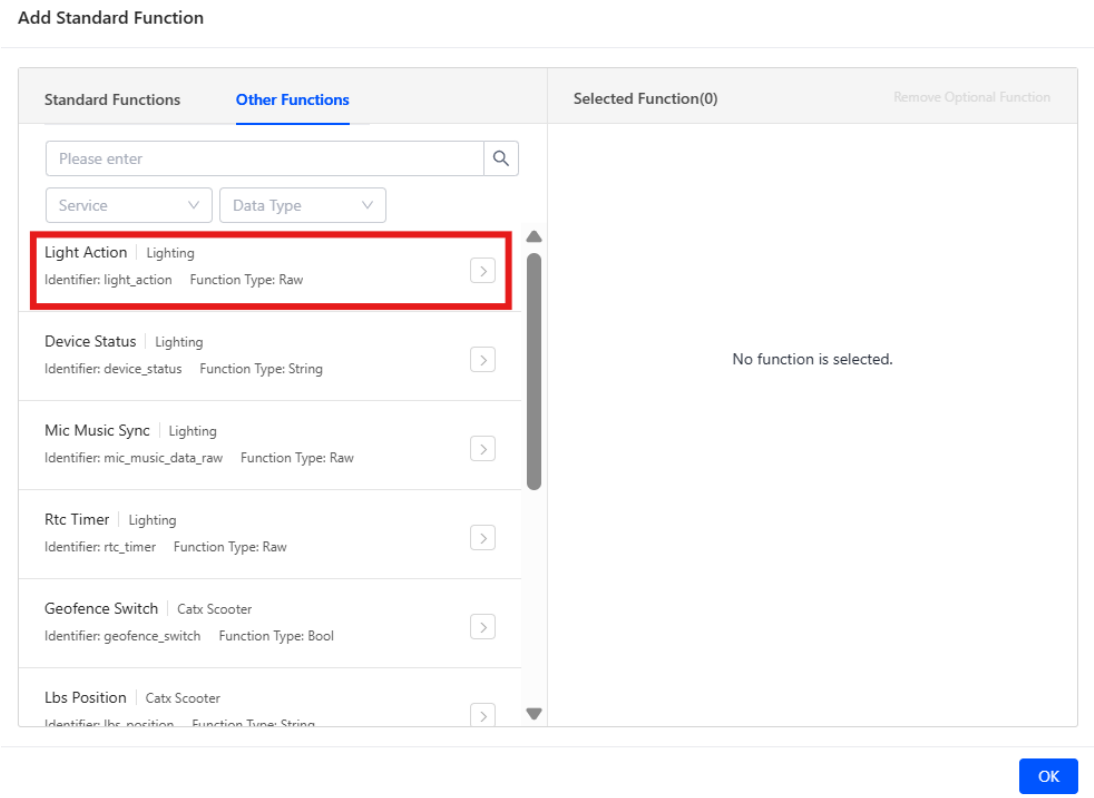

Figure 1.4: Adding the Light Action function to map cloud DP to device control.

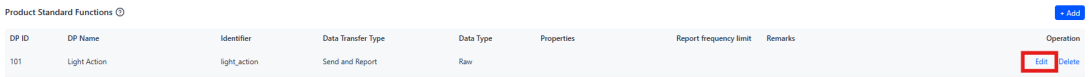

Figure 1.5: Checking that the function data type is Boolean for ESP32 compatibility.

5. **Licensing:**
   Navigate to the **Hardware Development** tab. Because this is a custom solution, you need to request free debugging licenses. Click **Get 2 Free Licenses** to generate licenses that allow testing the custom firmware. Ensure that the delivery mode is set to **License List** to receive the UUID and AuthKey in the correct format.

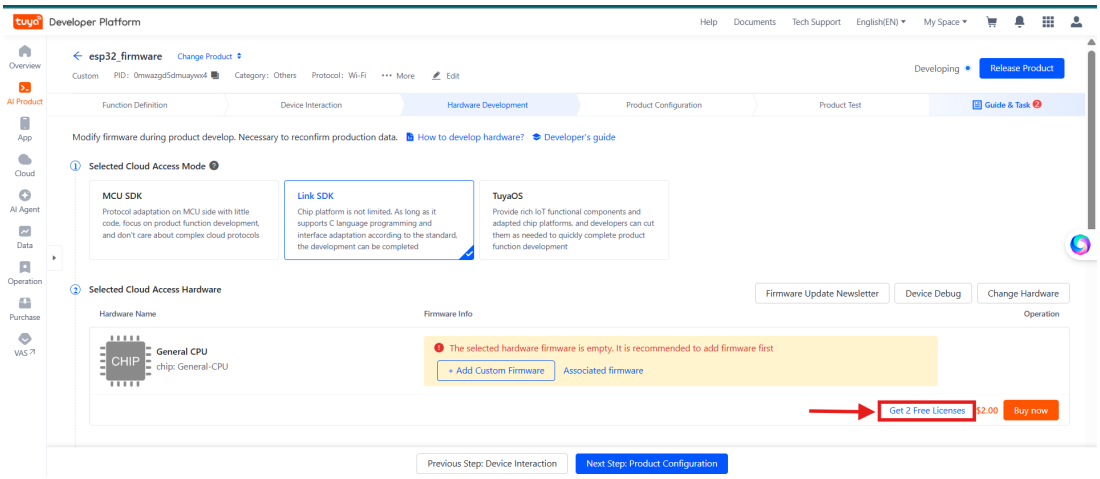

Figure 1.6: Accessing the Hardware Development tab to request licenses.

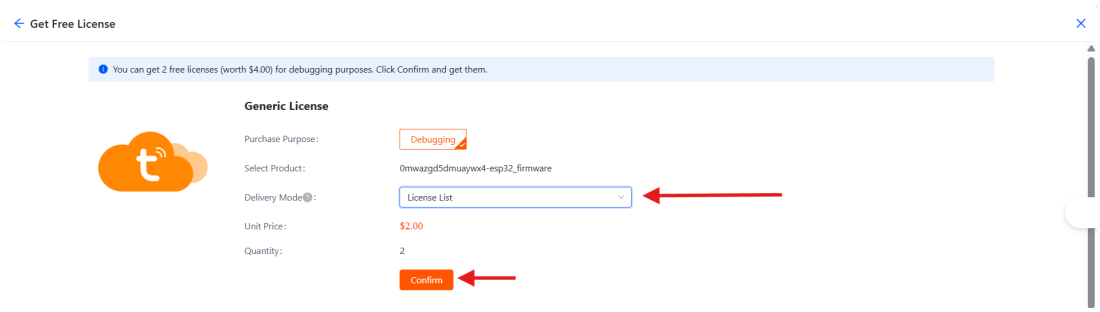

Figure 1.7: License delivery mode configured as "License List" to receive UUID/AuthKey.

6. **Order Submission:**
   Confirm the license request. The platform will generate a unique *UUID* and *AuthKey*, which are essential to embed in the firmware for cloud authentication.

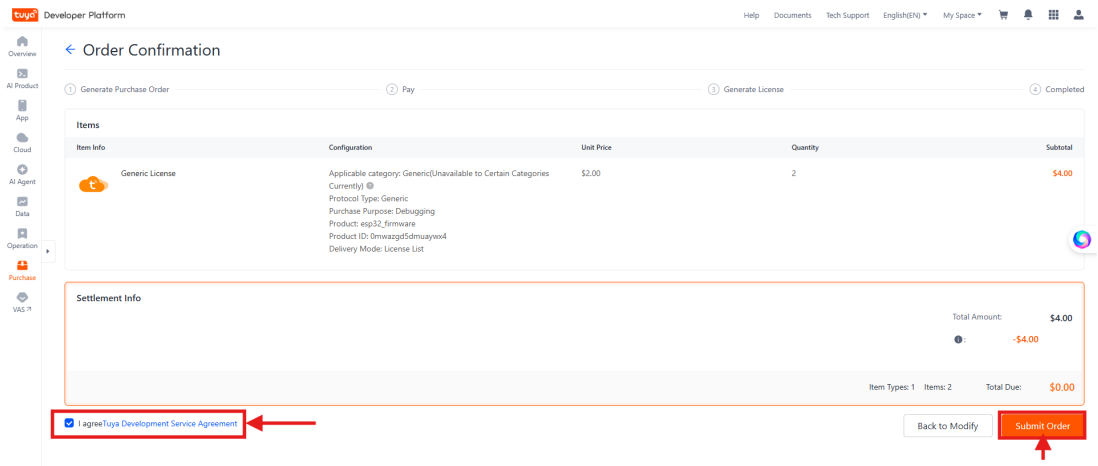

Figure 1.8: Submitting the license request and retrieving UUID/AuthKey.

1.2 Firmware Build Process

After obtaining the UUID and AuthKey, the firmware must be configured and compiled for the ESP32 board:

1. **GPIO Configuration:**
   Open the `main.c` file in the `switch_demo` project folder. Map the control logic to the appropriate GPIO (e.g., GPIO 14). This ensures the firmware can toggle the connected relay or light.

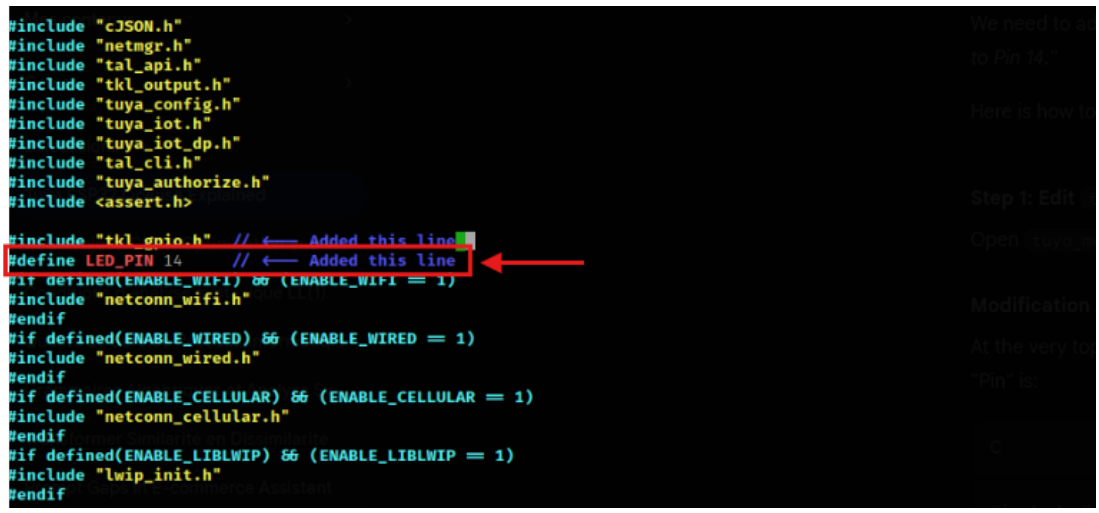

Figure 1.9: Mapping the control logic to GPIO 14 in the source code.

2. **Project Environment Initialization:**
   Open a terminal in the project folder and run the command:

   ```
   tos.py config choice
   ```

   This sets up the build environment and prepares the project for compilation.

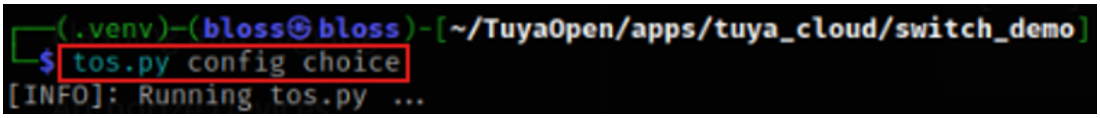

Figure 1.10: Initializing the project configuration using tos.py.

3. **Target Board Selection:**
   Choose the configuration file corresponding to your ESP32 board, for example `ESP32.config`. This step ensures the correct board settings and pin mappings are used during compilation.

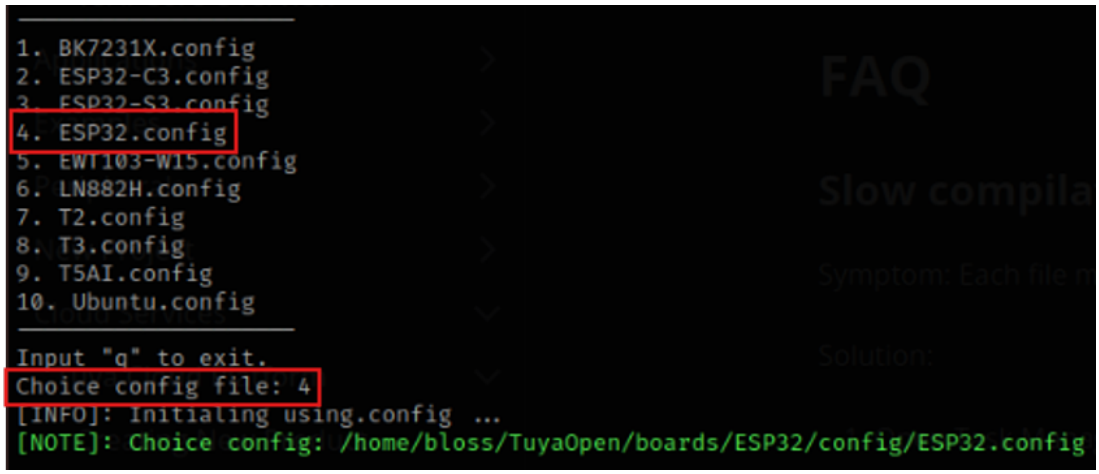

Figure 1.11: Selecting the ESP32 board configuration.

4. **Compilation:**
   Execute:

   ```
   tos.py build
   ```

   The source code will be compiled into a binary file. Verify that the build completes successfully and that no errors appear.

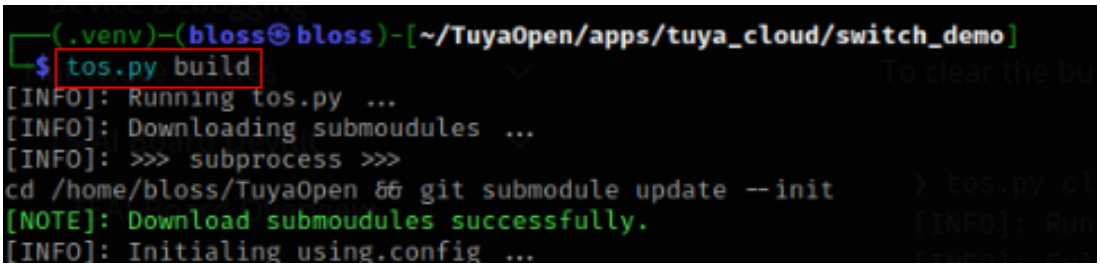

Figure 1.12: Compiling the firmware to generate the binary file.

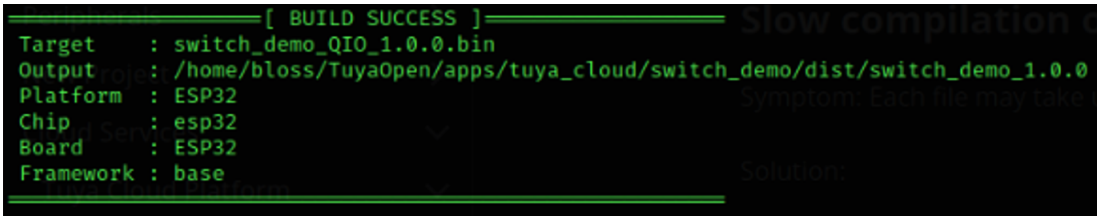

Figure 1.13: Successful build confirmation screen.

5. **Flashing the Board:**
   Connect the ESP32 board via USB and use the ESP-IDF flash tool to upload the compiled binary. Ensure the correct serial port is selected and the flashing process completes without errors.

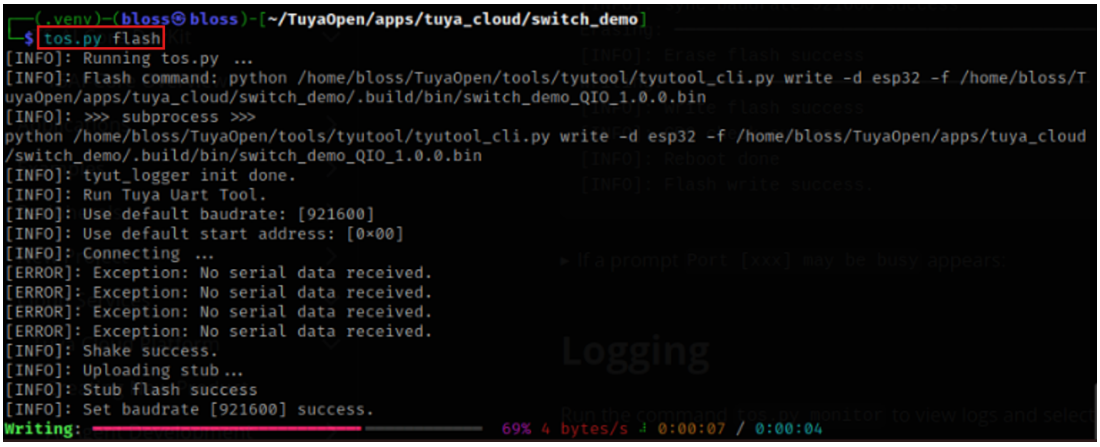

Figure 1.14: Uploading the firmware to the ESP32 using the flash tool.